Rainfall in NSW (SILO, 2000–2025)

This notebook turns the SILO `.nc` files in `data/rainfall_data/` into NSW rainfall maps and yearly totals, and builds a table of rainfall per Bird Atlas grid cell that can be joined to the sightings.

Each step has a short explanation, so this notebook doubles as the how-to for working with `.nc` files later. The 1960s files are skipped (and `1962.monthly_rain.nc` is an incomplete download).

Packages: `xarray` reads the files and `netCDF4` is the engine underneath. Both are installed in the `BirdWatching` conda env. On a new machine: `conda install -n BirdWatching -c conda-forge xarray netcdf4`.

What a `.nc` file is

NetCDF (`.nc`) stores **gridded data with labels**. Think of it as a stack of spreadsheets where every row, column and sheet is labelled with a real coordinate:

Term	In these files
dimensions	`time` (12 months), `lat` (681 rows), `lon` (841 columns)
coordinates	the labels on each dimension: dates, latitudes −44 to −10, longitudes 112 to 154 (0.05° ≈ 5 km steps)
data variable	`monthly_rain(time, lat, lon)`: total rainfall in mm for that month and grid square
attributes	metadata such as units, source and fill value

There is one file per year covering all of Australia. Ocean squares are missing (`NaN`). The numbers are stored as packed integers to save space, and xarray unpacks them to millimetres automatically.

`xarray` is like pandas or polars, except you select by coordinate name and value instead of row number.

```
import glob
import os

import numpy as np
import polars as pl
import xarray as xr
import plotly.graph_objects as go
import plotly.io as pio
from plotly.subplots import make_subplots

pio.renderers.default = "plotly_mimetype+notebook_connected"

RAIN_DIR = "data/rainfall_data"
```

Step 1: open one file and look at it

`xr.open_dataset` only reads the header at first, so this is instant even for big files. Displaying `ds` in a cell shows the dimensions, coordinates, variables and attributes.

```
ds = xr.open_dataset(f"{RAIN_DIR}/2022.monthly_rain.nc")
ds
```

xarray.Dataset

▸ Dimensions:	(**time**: 12, **lat**: 681, **lon**: 841)			
▾ Coordinates:				
time	(time)	datetime64[ns]	2022-01-16T12:00:00 ... 2022-12-...	
lat	(lat)	float64	-44.0 -43.95 -43.9 ... -10.05 -10.0	
lon	(lon)	float64	112.0 112.0 112.1 ... 153.9 154.0	
▾ Data variables:				
monthly_rain	(time, lat, lon)	float64	...	
crs	()	|S1	...	
▸ Attributes: (10)				

Selecting data uses **labels**, not positions:

- `.sel(time="2022-03")` picks a month by date
- `.sel(lat=-33.87, lon=151.21, method="nearest")` picks the grid square closest to a point (here, Sydney)
- `.sel(lat=slice(-37.7, -28.0))` picks a range. Latitudes are stored south-to-north, so the smaller (more negative) number goes first.

```
# Sydney's monthly rainfall in 2022 (mm). .values turns the result into a plain numpy array
sydney_2022 = ds["monthly_rain"].sel(lat=-33.87, lon=151.21, method="nearest")
sydney_2022.values.round(0)
```

```
array([161., 348., 551., 261., 181.,  16., 354.,  53.,  87., 277.,  58.,
        72.])
```

Step 2: load 2000–2025, cropped to NSW

The full Australian grid is about 700 MB for 26 years. Cropping each file to the NSW area **before** `.load()` reads only about 130 MB from disk. `xr.concat` then joins the years along the `time` dimension.

(xarray also has `xr.open_mfdataset` for opening many files at once, but it needs the extra `dask` package. The loop below does the same thing.)

```
# NSW bounding box (same as the bird heatmaps; the grid stops at 154°E)
NSW_LAT = (-37.7, -28.0)
NSW_LON = (140.5, 154.0)
FIRST_YEAR = 2000  # ignore the 1960s files

files = sorted(glob.glob(f"{RAIN_DIR}/*.monthly_rain.nc"))
files = [f for f in files if int(os.path.basename(f)[:4]) >= FIRST_YEAR]

monthly = xr.concat(
    [
        xr.open_dataset(f)["monthly_rain"]
        .sel(lat=slice(*NSW_LAT), lon=slice(*NSW_LON))
        .load()
        for f in files
    ],
    dim="time",
)
monthly
```

```
<xarray.DataArray 'monthly_rain' (time: 312, lat: 195, lon: 271)>  132 MB of monthly
rainfall, 2000–2025, NSW box
(full printout omitted from the PDF)
```

Step 3: keep only NSW

The bounding box also covers parts of Queensland, Victoria and South Australia. To keep only NSW, test each grid square against an outline of the state and set everything outside to `NaN` with `.where(...)`.

The outline below is **approximate** (accurate to roughly 10–20 km along the river borders) and includes the ACT. For an exact border, download the ABS state boundaries shapefile and use `geopandas` + `regionmask` instead.

```
# Approximate NSW border as (lon, lat) points, clockwise from the north-west corner.
# The east side runs out to sea; ocean squares are already NaN, so the coastline
comes from the data.
NSW_BORDER = [
    (141.00, -29.00), (148.98, -29.00),                          # straight 29°S line
    (150.00, -28.55), (150.35, -28.60), (151.15, -28.85),        # Barwon / Macintyre
/ Dumaresq rivers
    (151.95, -28.95), (152.10, -28.65), (152.50, -28.27),        # Great Dividing /
McPherson Ranges
    (153.10, -28.30), (153.55, -28.17),                          # Point Danger
    (154.00, -28.17), (154.00, -37.50), (149.98, -37.50),        # out to sea, back
to Cape Howe
    (148.20, -36.80),                                            # straight line to
the Murray's source
    (147.95, -36.05), (146.92, -36.08), (146.39, -36.00),        # Murray River:
Albury, Corowa
    (145.57, -35.81), (144.75, -36.13), (143.56, -35.34),        # Tocumwal, Echuca,
Swan Hill
    (142.77, -34.58), (142.15, -34.18), (141.92, -34.11),        # Robinvale,
Mildura, Wentworth
    (141.00, -34.00),                                            # up the 141°E line
]
```

```python
def inside_polygon(lon, lat, polygon):
    # Ray casting: a point is inside if a ray going east crosses the border an odd number of times
    inside = np.zeros(lon.shape, dtype=bool)
    xs, ys = np.array(polygon).T
    for x1, y1, x2, y2 in zip(xs, ys, np.roll(xs, -1), np.roll(ys, -1)):
        crosses = (y1 > lat) != (y2 > lat)
        with np.errstate(divide="ignore", invalid="ignore"):
            x_cross = x1 + (lat - y1) * (x2 - x1) / (y2 - y1)
        inside ^= crosses & (lon < x_cross)
    return inside

lon_grid, lat_grid = np.meshgrid(monthly["lon"], monthly["lat"])
in_nsw = xr.DataArray(
    inside_polygon(lon_grid, lat_grid, NSW_BORDER),
    coords={"lat": monthly["lat"], "lon": monthly["lon"]},
    dims=("lat", "lon"),
)
monthly = monthly.where(in_nsw)
```

Step 4: monthly → yearly totals

`groupby("time.year").sum()` adds up the 12 months of each year, the same way `group_by` works in polars.

`min_count=12` matters: by default xarray treats missing values as 0, so ocean squares would get 0 mm instead of `NaN`. Requiring all 12 months keeps them missing.

```python
annual = monthly.groupby("time.year").sum(min_count=12)   # dims: (year, lat, lon), mm per year
mean_annual = annual.mean("year")                          # average year, 2000–2025

# Grid squares get narrower further south, so weight by cos(latitude) when averaging over an area
area_weights = np.cos(np.deg2rad(annual["lat"]))
nsw_yearly = annual.weighted(area_weights).mean(["lat", "lon"]).to_series()
nsw_yearly.round(0)
```

```
year
2000    621.0
2001    479.0
2002    314.0
2003    485.0
2004    492.0
2005    497.0
2006    348.0
2007    549.0
2008    522.0
2009    492.0
2010    831.0
2011    672.0
2012    568.0
2013    468.0
2014    473.0
2015    548.0
2016    665.0
2017    454.0
2018    335.0
2019    252.0
2020    649.0
2021    736.0
2022    882.0
2023    436.0
2024    596.0
2025    541.0
Name: monthly_rain, dtype: float64
```

Step 5: charts

All charts share the styling below. Rainfall amount uses a single blue scale (light = little rain, dark = a lot). Wetter or drier than average uses red ↔ grey ↔ blue, with grey meaning "about average".

```
INK, INK_2, MUTED, GRID = "#0b0b0b", "#52514e", "#898781", "#e1e0d9"
SURFACE = "#fcfcfb"
BLUES = ["#cde2fb", "#9ec5f4", "#6da7ec", "#3987e5", "#256abf", "#184f95", "#0d366b"]
DRY, WET, NEUTRAL = "#e34948", "#2a78d6", "#f0efec"

# Grid squares are 0.05° on each side, but a degree of longitude is shorter than a
degree of latitude
# this far south. Stretch the y-axis so the map isn't squashed.
LAT_STRETCH = 1 / np.cos(np.deg2rad(np.mean(NSW_LAT)))

border_lon, border_lat = zip(*(NSW_BORDER + NSW_BORDER[:1]))
border_lon, border_lat = list(border_lon), list(border_lat)
# Drop the out-to-sea segment so the outline only traces land borders
sea = [i for i, x in enumerate(border_lon) if x == 154.0]
for i in reversed(sea):
    border_lon[i] = border_lat[i] = None

def border_trace():
    return go.Scatter(x=border_lon, y=border_lat, mode="lines",
line=dict(color=INK_2, width=1),
                      hoverinfo="skip", showlegend=False)

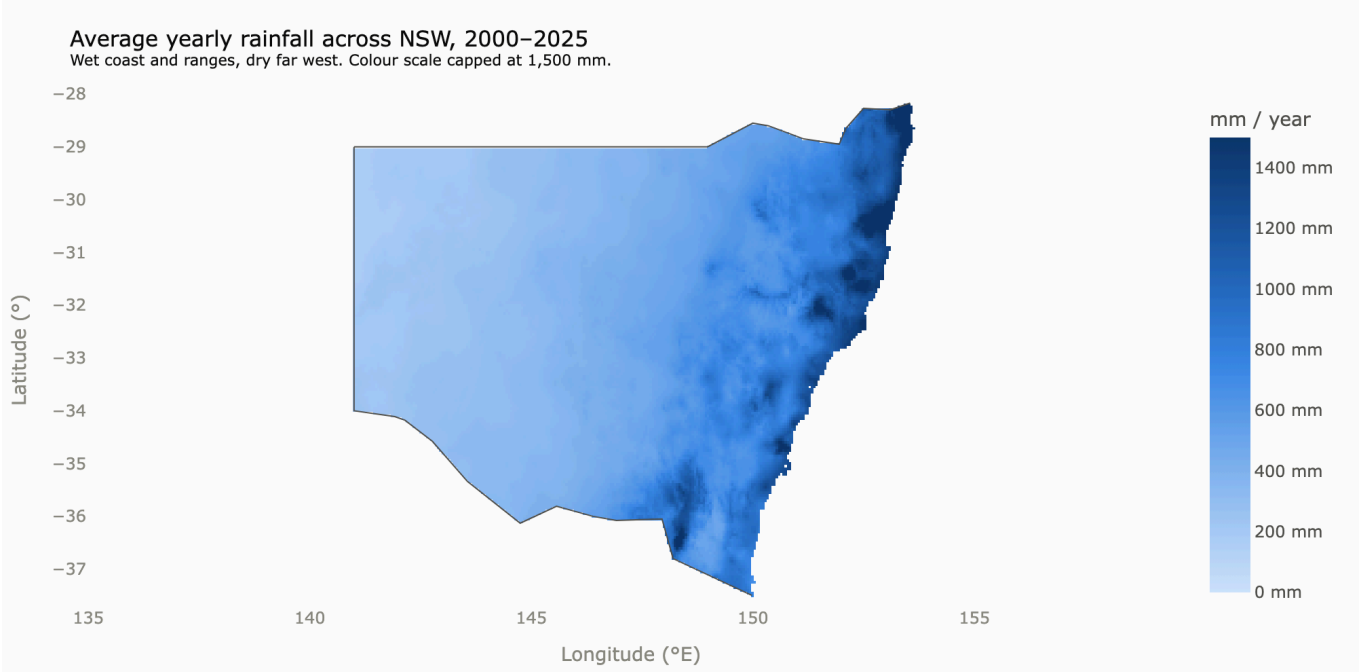
def style_map(fig, title, height=560):
```

```
    fig.update_layout(
        title=dict(text=title, font=dict(size=16, color=INK)), height=height,
        plot_bgcolor=SURFACE, paper_bgcolor=SURFACE, font=dict(color=INK_2),
        margin=dict(l=50, r=20, t=70, b=40),
    )
    fig.update_xaxes(title="Longitude (°E)", showgrid=False, zeroline=False,
color=MUTED,
                     range=[NSW_LON[0], NSW_LON[1]])
    fig.update_yaxes(title="Latitude (°)", showgrid=False, zeroline=False,
color=MUTED,
                     scaleanchor="x", scaleratio=LAT_STRETCH, range=[NSW_LAT[0],
NSW_LAT[1]])
    return fig
```

Average yearly rainfall, 2000–2025

`go.Heatmap` takes the grid directly: `x` = longitudes, `y` = latitudes, `z` = the 2-D array of values.

```
fig = go.Figure(go.Heatmap(
    x=mean_annual["lon"], y=mean_annual["lat"], z=mean_annual.values,
    colorscale=BLUES, zmin=0, zmax=1500,
    colorbar=dict(title="mm / year", ticksuffix=" mm", outlinewidth=0),
    hovertemplate="%{y:.2f}°, %{x:.2f}°E<br><b>%{z:.0f} mm</b> per year<extra>
</extra>",
))
fig.add_trace(border_trace())
style_map(fig, "Average yearly rainfall across NSW, 2000–2025<br>"
               "<sup>Wet coast and ranges, dry far west. Colour scale capped at 1,500
mm.</sup>")
fig.show()
```



Rainfall zones (dry / low / moderate / high)

These are the four groups from the research notes. `np.digitize` sorts each square into a band. The thresholds are a choice, so change `ZONE_EDGES` to try different cut-offs.

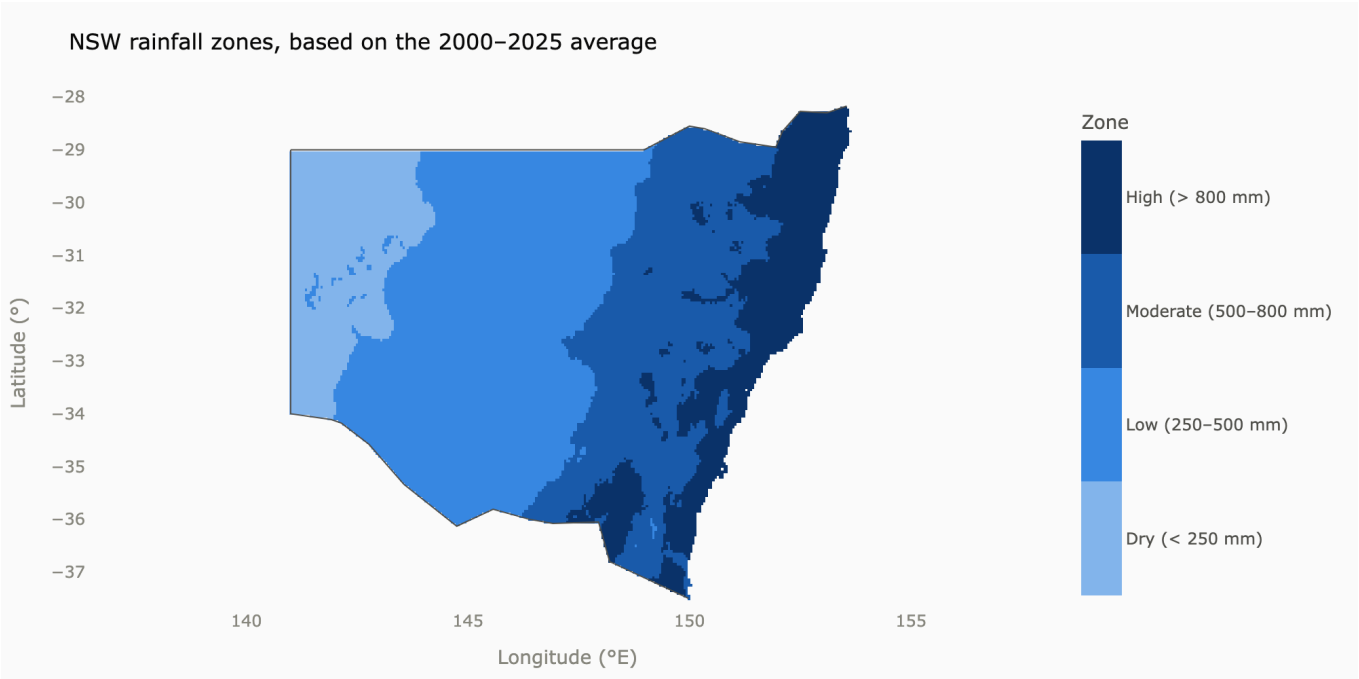
```
ZONE_EDGES = [250, 500, 800]                        # mm/year boundaries between zones
ZONE_NAMES = ["Dry (< 250 mm)", "Low (250–500 mm)", "Moderate (500–800 mm)", "High (>
800 mm)"]
ZONE_COLOURS = ["#86b6ef", "#3987e5", "#1c5cab", "#0d366b"]

zone = xr.where(mean_annual.notnull(), np.digitize(mean_annual, ZONE_EDGES), np.nan)

# Share of NSW's land area in each zone (area-weighted)
weights = (zone.notnull() * area_weights)
zone_share = pl.DataFrame({
    "zone": ZONE_NAMES,
    "share_of_nsw_%": [round(float((weights * (zone == k)).sum() / weights.sum() *
100), 1) for k in range(4)],
})

# A heatmap with stepped colours: each zone value k fills the band [k/4, (k+1)/4] of
the colour scale
steps = []
for k, c in enumerate(ZONE_COLOURS):
    steps += [[k / 4, c], [(k + 1) / 4, c]]

fig = go.Figure(go.Heatmap(
    x=zone["lon"], y=zone["lat"], z=zone.values, zmin=-0.5, zmax=3.5,
    colorscale=steps,
    customdata=mean_annual.values,
    text=np.array(ZONE_NAMES + [""], dtype=object)[np.nan_to_num(zone.values,
nan=4).astype(int)],
    colorbar=dict(tickvals=[0, 1, 2, 3], ticktext=ZONE_NAMES, outlinewidth=0,
title="Zone"),
    hovertemplate="%{y:.2f}°, %{x:.2f}°E<br><b>%{text}</b><br>%{customdata:.0f} mm
per year<extra></extra>",
))
fig.add_trace(border_trace())
style_map(fig, "NSW rainfall zones, based on the 2000–2025 average")
fig.show()
zone_share
```



shape: (4, 2)

zone	share_of_nsw_%
str	f64
"Dry (< 250 mm)"	14.4
"Low (250–500 mm)"	42.3
"Moderate (500–800 mm)"	26.9
"High (> 800 mm)"	16.4

Year by year: how wet was NSW?

This is the area-weighted NSW average for each year. Bars above the dashed line were wetter than the 2000–2025 average.

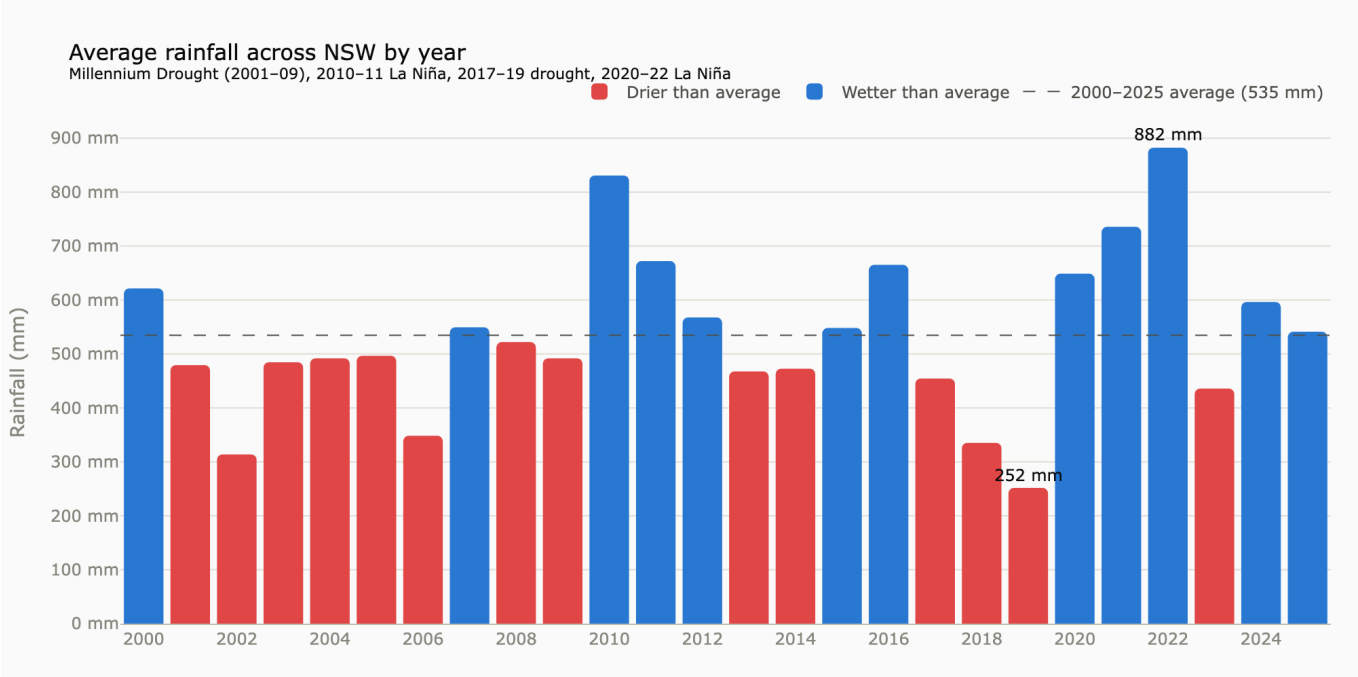
```
avg = nsw_yearly.mean()
years, totals = nsw_yearly.index.to_numpy(), nsw_yearly.to_numpy()
wetter = totals >= avg

fig = go.Figure()
for mask, colour, name in [(~wetter, DRY, "Drier than average"), (wetter, WET,
"Wetter than average")]:
    fig.add_trace(go.Bar(
        x=years[mask], y=totals[mask], name=name, marker=dict(color=colour,
line=dict(width=0)),
        hovertemplate="<b>%{x}</b><br>%{y:.0f} mm<extra>" + name + "</extra>",
    ))
# Average as a dashed line with its own legend entry (a label on the plot would cover
a bar)
fig.add_trace(go.Scatter(
    x=[years[0] - 0.5, years[-1] + 0.5], y=[avg, avg], mode="lines",
    line=dict(color=INK_2, width=1, dash="dash"), name=f"2000–2025 average ({avg:.0f}
mm)",
    hoverinfo="skip",
))

# Label only the extremes
for yr in [int(nsw_yearly.idxmin()), int(nsw_yearly.idxmax())]:
    fig.add_annotation(x=yr, y=nsw_yearly[yr], text=f"{nsw_yearly[yr]:.0f} mm",
showarrow=False,
                       yshift=10, font=dict(color=INK, size=12))

fig.update_layout(
    title=dict(text="Average rainfall across NSW by year<br>"
                    "<sup>Millennium Drought (2001–09), 2010–11 La Niña, 2017–19
drought, 2020–22 La Niña</sup>",
               font=dict(size=16, color=INK)),
    barmode="overlay", bargap=0.15, barcornerradius=4, height=440,
    plot_bgcolor=SURFACE, paper_bgcolor=SURFACE, font=dict(color=INK_2),
    legend=dict(orientation="h", y=1.02, x=1, xanchor="right", yanchor="bottom"),
    margin=dict(l=60, r=20, t=90, b=40),
)
fig.update_xaxes(dtick=2, showgrid=False, color=MUTED, linecolor="#c3c2b7")
fig.update_yaxes(title="Rainfall (mm)", gridcolor=GRID, zeroline=False, color=MUTED,
```

```
ticksuffix=" mm")
fig.show()
```



Driest vs wettest year compared with average

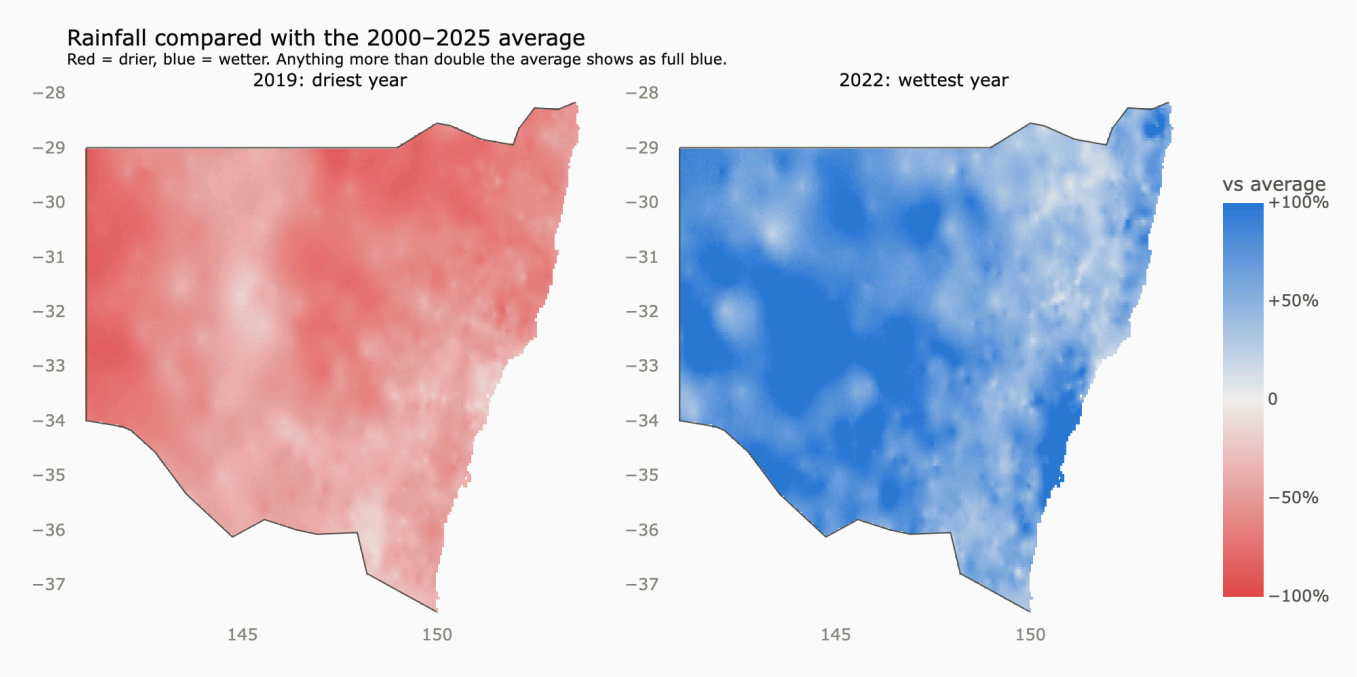
Anomaly = that year ÷ the 2000–2025 average − 1, shown as a percentage. Because the data is labelled, `annual.sel(year=2019) / mean_annual` divides every grid square by its own average in one line.

```
anomaly = (annual / mean_annual - 1) * 100       # % above (+) or below (−) average, every year and square
driest, wettest = int(nsw_yearly.idxmin()), int(nsw_yearly.idxmax())

fig = make_subplots(rows=1, cols=2, horizontal_spacing=0.06,
                    subplot_titles=[f"{driest}: driest year", f"{wettest}: wettest year"])
for col, yr in [(1, driest), (2, wettest)]:
    a = anomaly.sel(year=yr)
    fig.add_trace(go.Heatmap(
        x=a["lon"], y=a["lat"], z=a.values, coloraxis="coloraxis",
        customdata=annual.sel(year=yr).values,
        hovertemplate=f"<b>{yr}</b> · " + "%{y:.2f}°, %{x:.2f}°E<br>%{z:+.0f}% vs average"
                      "<br>%{customdata:.0f} mm<extra></extra>",
    ), row=1, col=col)
    fig.add_trace(border_trace(), row=1, col=col)

fig.update_layout(coloraxis=dict(
    colorscale=[[0, DRY], [0.5, NEUTRAL], [1, WET]], cmin=-100, cmax=100,
    colorbar=dict(title=dict(text="vs average", side="top"), len=0.85, y=0.45,
                  tickvals=[-100, -50, 0, 50, 100],
                  ticktext=["−100%", "−50%", "0", "+50%", "+100%"], outlinewidth=0),
))
style_map(fig, "Rainfall compared with the 2000–2025 average<br>"
               "<sup>Red = drier, blue = wetter. Anything more than double the average shows as full blue.</sup>",
          height=480)
```

```
fig.update_yaxes(scaleanchor=None)                    # subplots: keep panels the same
size instead
fig.update_xaxes(title=None)
fig.update_yaxes(title=None)
fig.update_annotations(font=dict(color=INK, size=13))
fig.show()
```



Step 6: rainfall per Bird Atlas grid cell

The sightings are located by 10-minute Bird Atlas grid cells (`key_latitude` , `key_longitude` , which are the cell centres). Each 10-minute cell holds about 10 SILO squares, so the steps are:

1. Work out which atlas cell each SILO square falls in. Cell edges are on whole multiples of 10 minutes, so the centre is the 10-minute block plus 5 minutes.
2. Average the squares in each cell for each year.

The result is a polars table with one row per (cell, year) that can be joined onto the sightings.

```
def to_atlas_cell(deg: pl.Expr) -> pl.Expr:
    # Decimal degrees -> centre of the 10-minute cell it falls in (keeps the sign)
    minutes = deg.abs() * 60
    centre = ((minutes // 10) * 10 + 5) / 60
    return (centre * deg.sign()).round(4)

cell_rain = (
    pl.from_pandas(annual.to_dataframe(name="rain_mm").reset_index())
    .drop_nulls("rain_mm")
    .with_columns(key_latitude=to_atlas_cell(pl.col("lat")),
key_longitude=to_atlas_cell(pl.col("lon")))
    .group_by("key_latitude", "key_longitude", "year")
    .agg(pl.col("rain_mm").mean().round(1))
    .sort("key_latitude", "key_longitude", "year")
)
cell_rain
```

shape: (74_490, 4)

key_latitude	key_longitude	year	rain_mm
f64	f64	i64	f64
-37.5833	150.0833	2000	755.1
-37.5833	150.0833	2001	797.7
-37.5833	150.0833	2002	794.5
-37.5833	150.0833	2003	750.6
-37.5833	150.0833	2004	713.8
…	…	…	…
-28.25	153.5833	2021	2133.1
-28.25	153.5833	2022	2640.9
-28.25	153.5833	2023	1186.0
-28.25	153.5833	2024	2519.3
-28.25	153.5833	2025	2379.0

```
# Save the table so birds_python.ipynb can load it without redoing the .nc processing
os.makedirs("outputs", exist_ok=True)
cell_rain.write_parquet("outputs/cell_rain.parquet")
```

Linking to the sightings in `birds_python.ipynb`

The cell above saves the table to `outputs/cell_rain.parquet`. In `birds_python.ipynb`, after `all_obs` is built, add:

```
cell_rain = pl.read_parquet("outputs/cell_rain.parquet")

obs_with_rain = (
    all_obs
    .with_columns(
        # Round to 4 decimals so the decimal-degree cell centres match
exactly
        pl.col("key_latitude", "key_longitude").round(4),
        year=pl.col("Observation End Date").dt.year(),
    )
    .join(cell_rain, on=["key_latitude", "key_longitude", "year"],
how="left")
)
```

Each sighting then gets a `rain_mm` column: the total rainfall in its grid cell for that calendar year.

When tested on `outputs/all_obs.csv`, **99% of the 2.48 million sightings from 2000–2025 matched**. The rest are expected:

- Lord Howe Island (159°E) and offshore cells: the SILO grid stops at 154°E and has no ocean data
- A few cells right on the Queensland/Victoria border that fall outside the approximate NSW outline

Sightings from before 2000, and DES sightings from 2026 (there's no 2026 rainfall file yet), also get a missing `rain_mm`.

Cheat sheet for next time

```
import xarray as xr

ds = xr.open_dataset("file.nc")              # open (reads header only)
ds                                           # see dims, coords, variables,
attributes
da = ds["monthly_rain"]                      # pick a variable -> DataArray

da.sel(time="2022-03")                       # select by label
da.sel(lat=-33.87, lon=151.21, method="nearest")   # nearest point
da.sel(lat=slice(-37.7, -28), lon=slice(140.5, 154))  # a box (lat south ->
north)
da.isel(time=0)                              # select by position instead

xr.concat([a, b], dim="time")                # stack files along time
da.groupby("time.year").sum(min_count=12)    # monthly -> yearly totals
da.groupby("time.month").mean()              # average for each calendar
month (seasonality)
da.mean("time")                              # average over a dimension
da.where(mask)                               # blank out squares where mask
is False

da.values                                    # -> numpy array
da.to_dataframe(name="rain_mm").reset_index()   # -> pandas (then
pl.from_pandas for polars)
```

Getting more years: SILO publishes one file per year and variable at `https://s3-ap-southeast-2.amazonaws.com/silo-open-data/Official/annual/monthly_rain/<YEAR>.monthly_rain.nc`. The same pattern works for other variables such as `max_temp` or `daily_rain`, with the folder and file name changed to match. Save new files into `data/rainfall_data/` and change `FIRST_YEAR` in Step 2.

Gotchas

- Latitudes run south → north, so `slice(-37.7, -28)`. A reversed slice silently returns nothing.
- Always pass `min_count` when summing, or ocean and missing squares become 0 mm.
- Crop with `.sel` **before** `.load()`, or every file is read in full.
- When averaging over an area, weight by `cos(latitude)`.